

## Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An autonomous Institution affiliated to Anna University)

|                     |                                                  |          |            |
|---------------------|--------------------------------------------------|----------|------------|
| Degree & Branch     | B.E Computer Science & Engineering               | Semester | VI         |
| Subject Code & Name | UCS2612 & Machine Learning Algorithms Laboratory |          |            |
| Academic year       | 2025-2026 Even                                   | Batch    | 2023-2027  |
| Name                | Kamalesh A                                       | Section  | CSE-B      |
| Registration No.    | 3122235001063                                    | Date     | 26-01-2026 |

### Experiment 2: Binary Classification using Naïve Bayes and K-Nearest Neighbors

#### 1.Aim and Objective:

##### Aim:

To implement Naïve Bayes and K-Nearest Neighbors (KNN) classifiers for a binary classification problem and evaluate their performance using multiple metrics and visualization techniques.

##### Objectives:

- To train Gaussian, Multinomial, and Bernoulli Naïve Bayes classifiers.
- To build a baseline KNN classifier and tune its hyperparameters using 5-Fold Cross-Validation.
- To compare KDTree and BallTree neighbor search strategies.
- To visualize model performance using confusion matrices, ROC curves, and accuracy plots.
- To analyze overfitting, underfitting, and bias–variance trade-offs.

#### 2. Dataset Description

The dataset consists of numerical features representing word frequencies, character frequencies, and capital letter usage statistics extracted from emails. The target variable class indicates whether an email is spam (1) or non-spam (0).

- **Type:** Binary classification
- **Features:** 57 numeric attributes

- **Target:** class
- **Domain:** Email spam detection

### 3. Preprocessing Steps:

The following preprocessing operations were performed:

- Checked the dataset for missing values.
- Removed records containing missing entries to ensure data quality.
- Separated features and target labels.
- Split the dataset into training, validation, and test sets using stratified sampling.
- Applied StandardScaler to normalize features for distance-based algorithms.
- Used unscaled data for Multinomial and Bernoulli Naïve Bayes models.

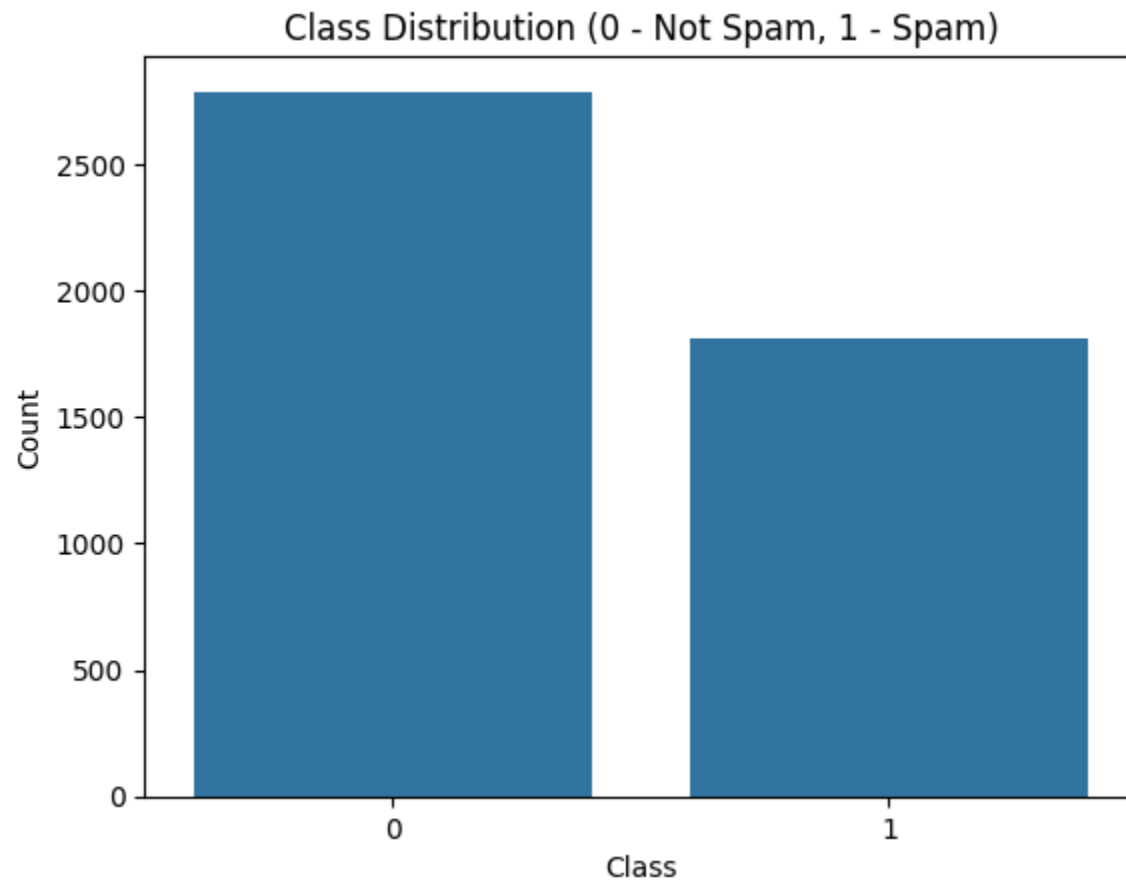
### 4. Implementation Details

The implementation included:

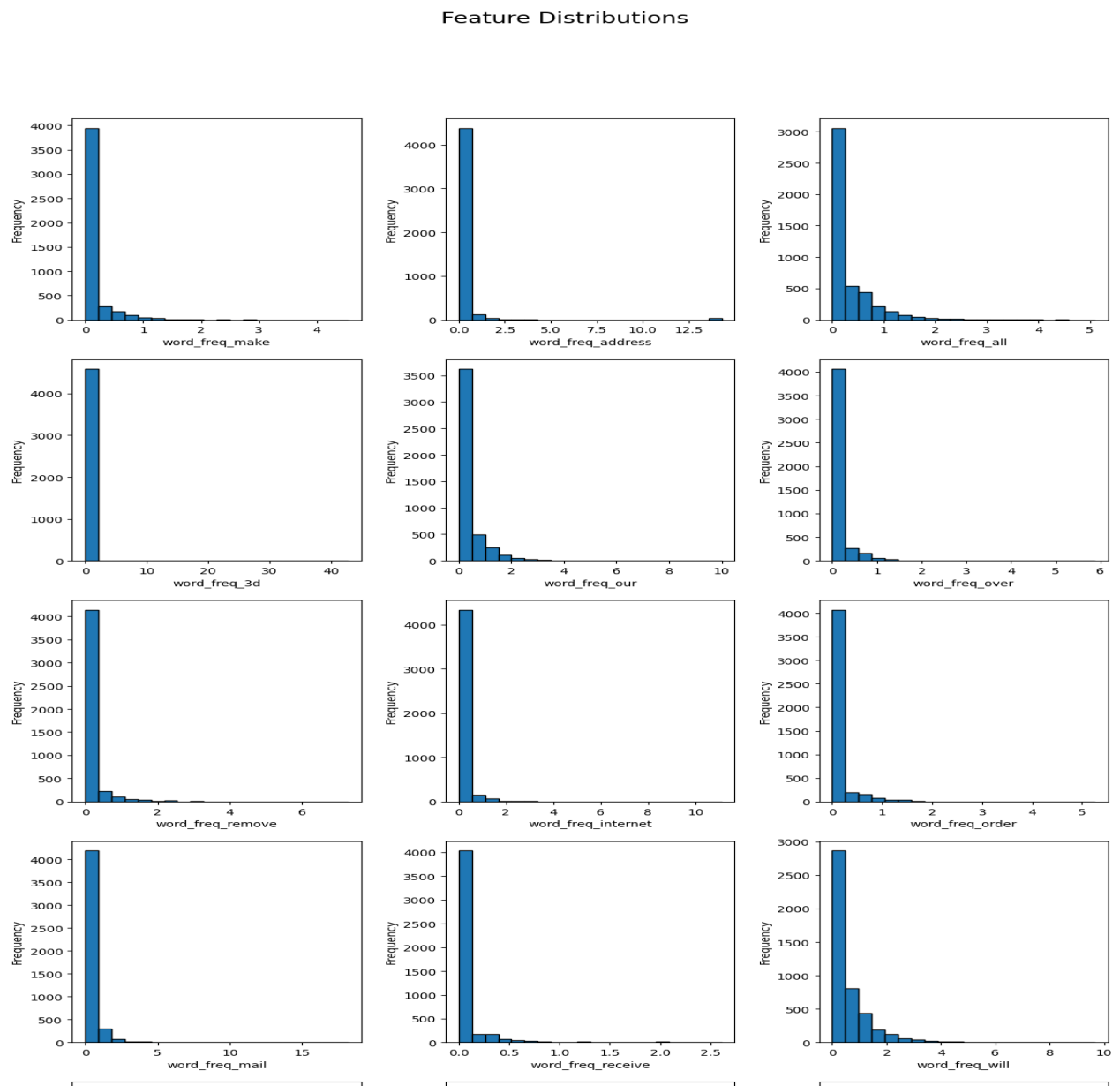
- Training three Naïve Bayes variants: GaussianNB, MultinomialNB, and BernoulliNB.
- Building a baseline KNN classifier.
- Performing hyperparameter tuning using GridSearchCV and RandomizedSearchCV with 5-fold cross-validation.
- Training optimized KNN models using KDTree and BallTree algorithms.
- Evaluating all models on the held-out test set.

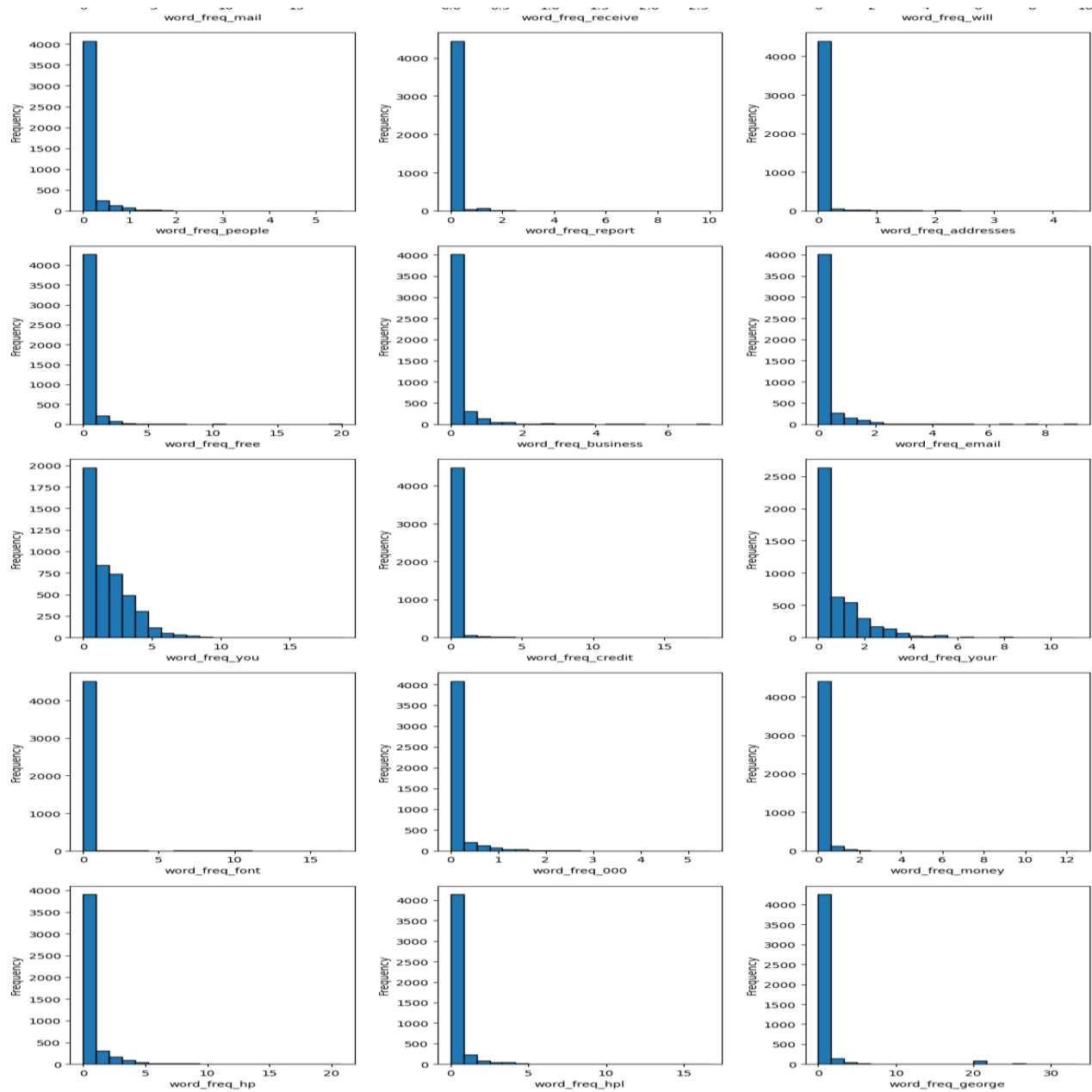
## 5. Visualizations

**Figure 1:** Class distribution plot.

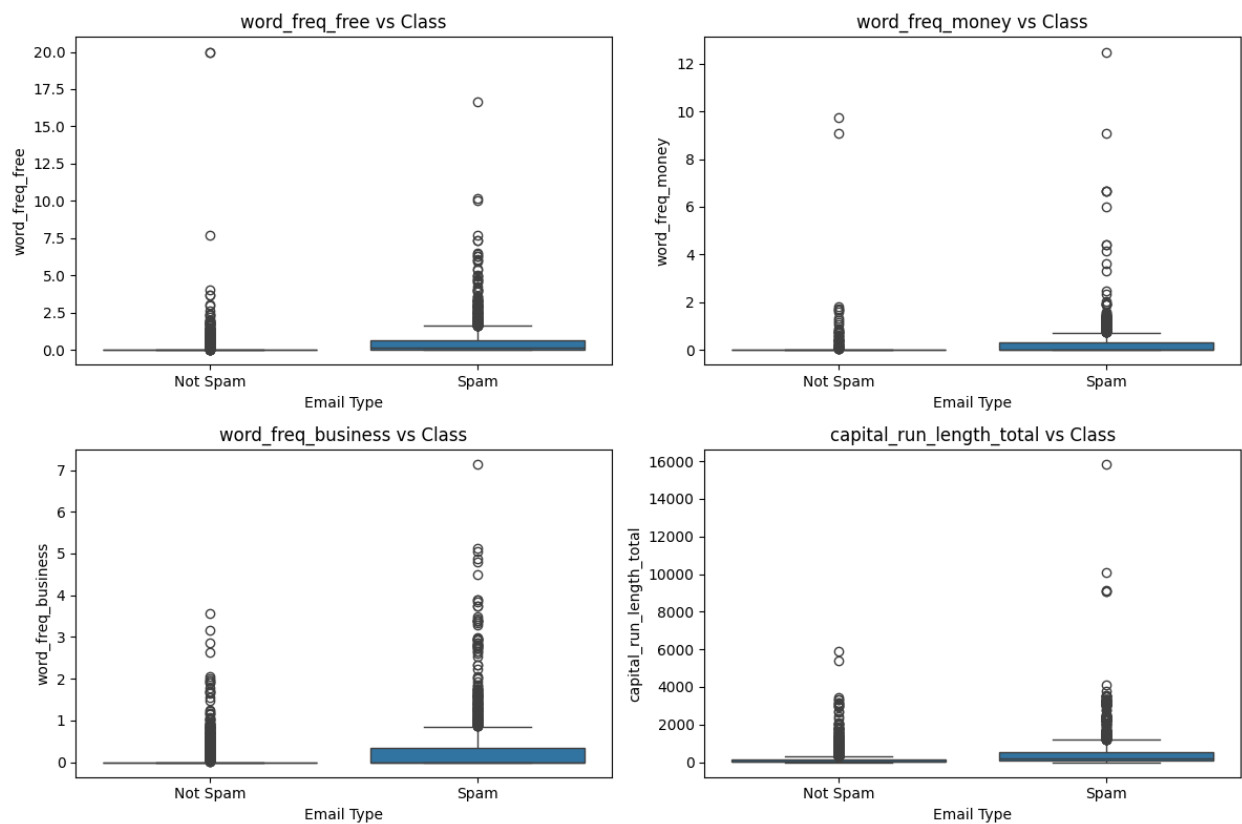


**Figure 2 &3:** Feature distribution histograms.

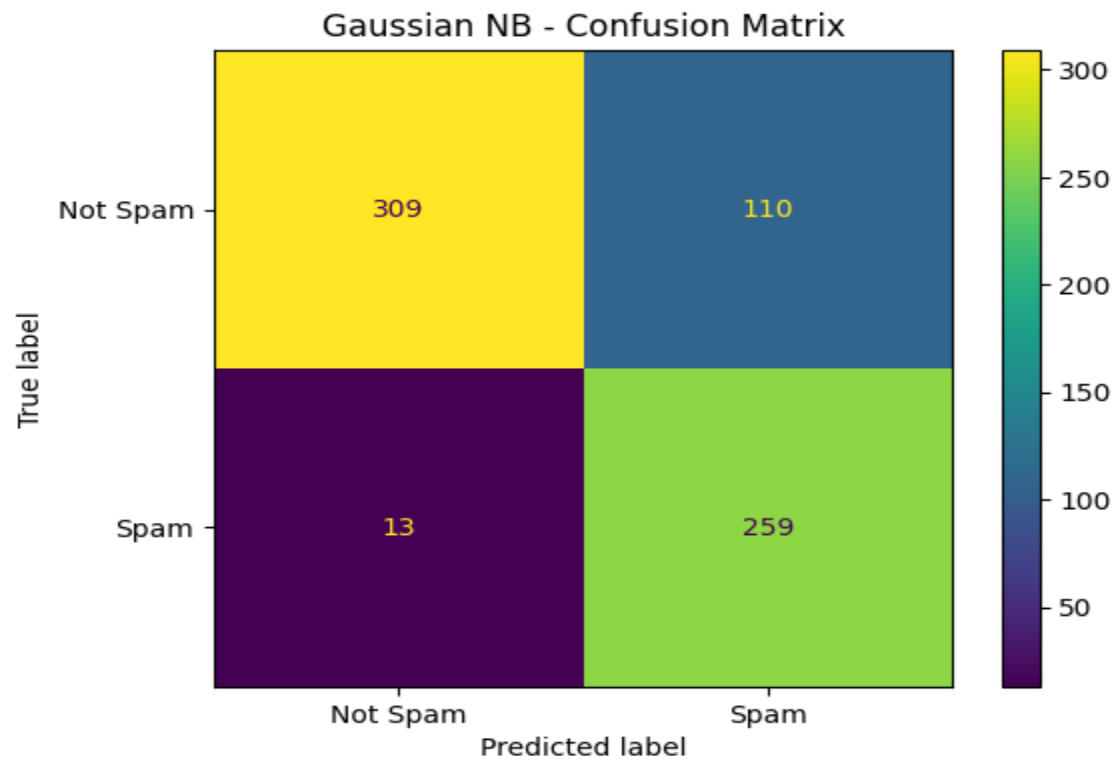




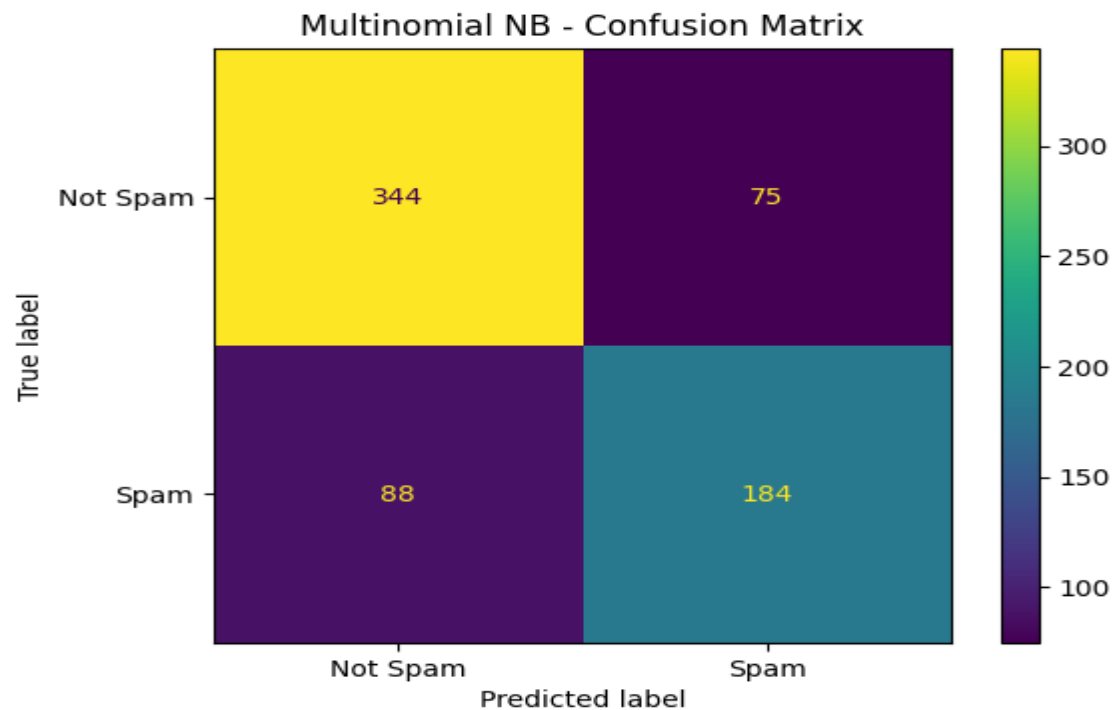
**Figure 4:** Boxplots of important features.



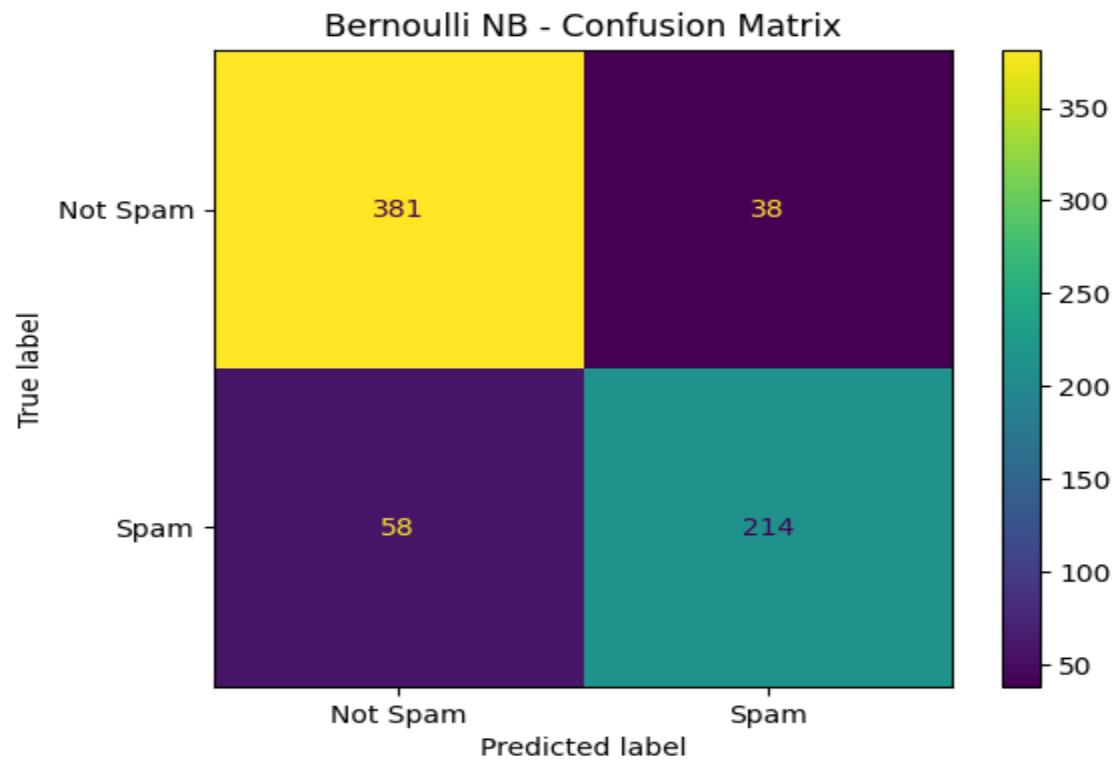
**Figure 5:** Confusion matrix for Gaussian NB



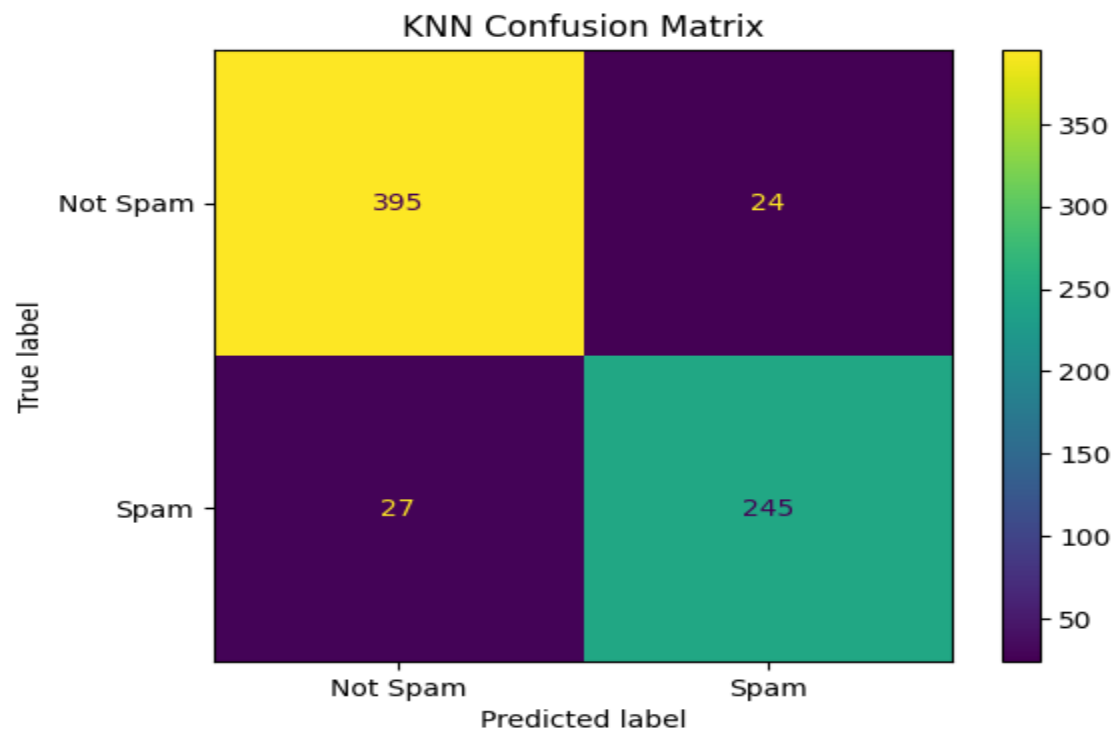
**Figure 6:** Confusion matrix for Multinomial NB



**Figure 7:** Confusion matrix for Bernoulli NB

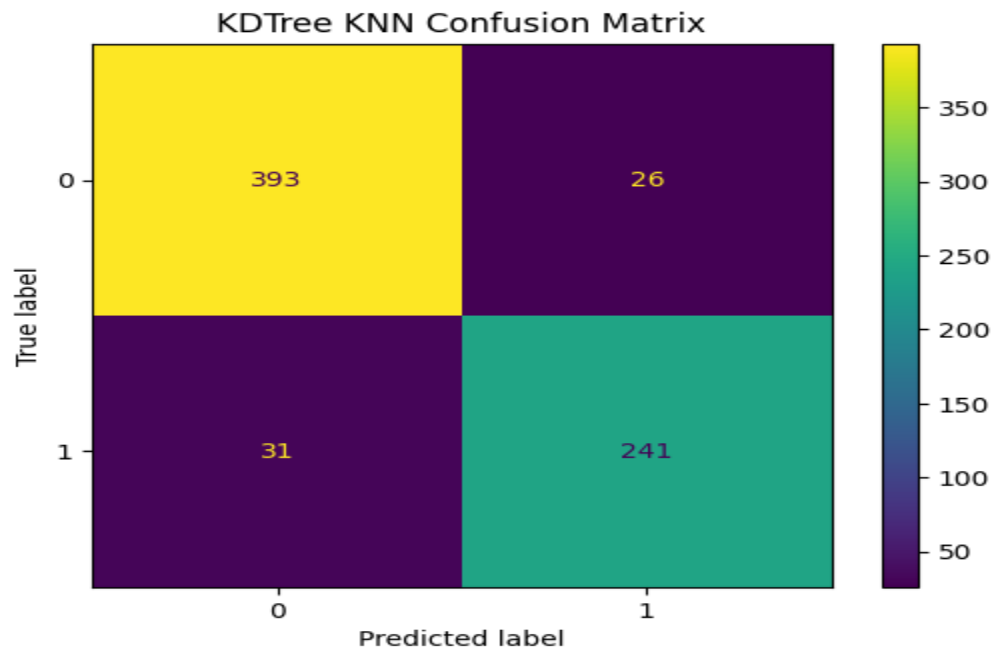


**Figure 8:** Confusion matrix for KNN

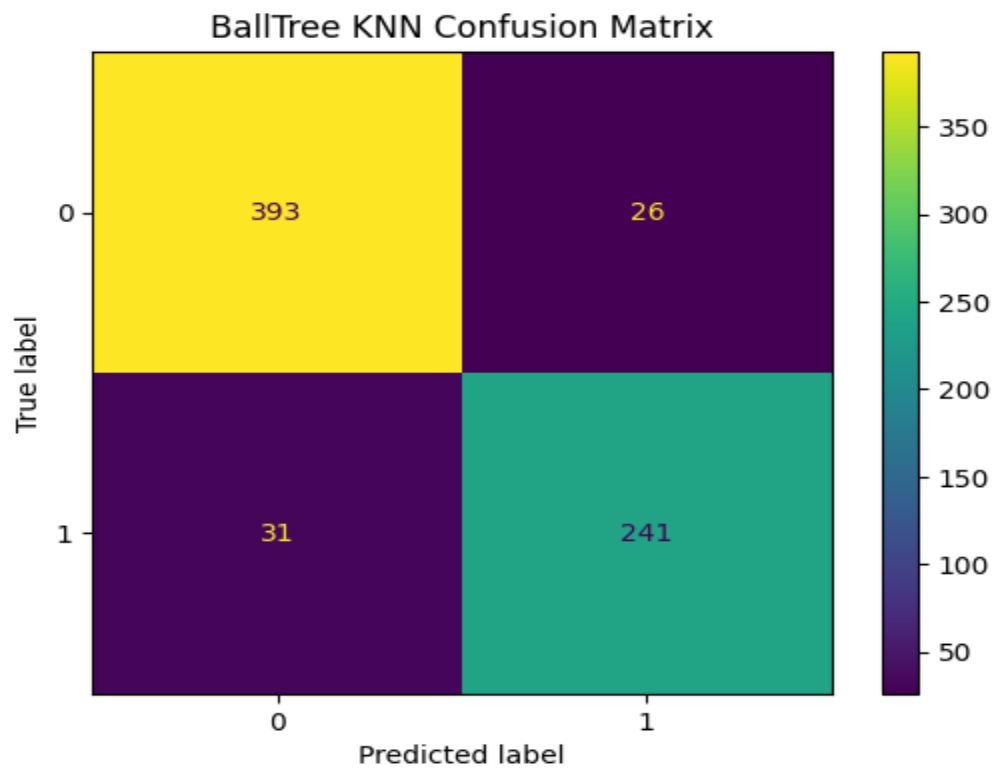




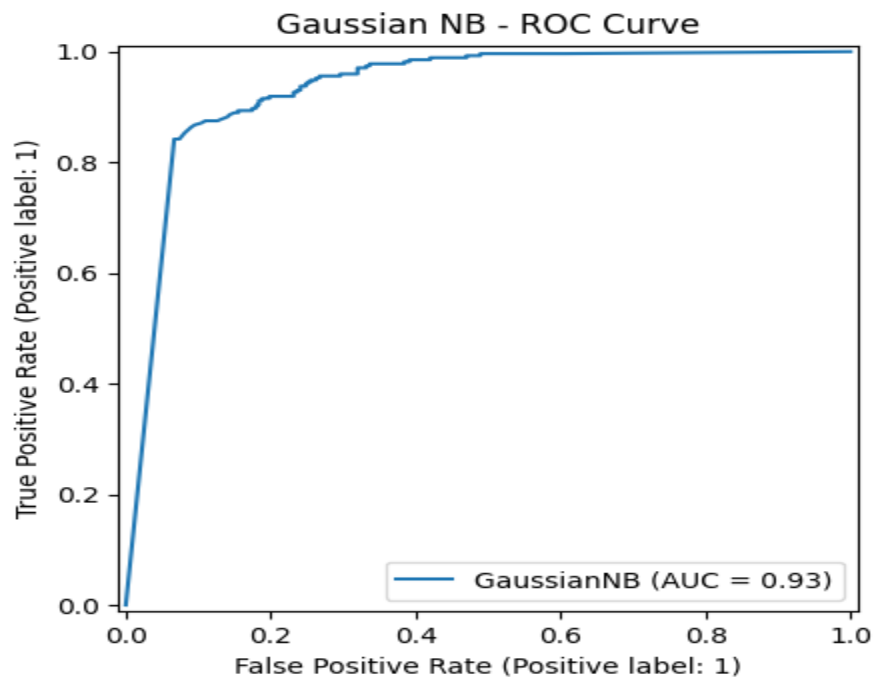
**Figure 9:** Confusion matrix for KD Tree



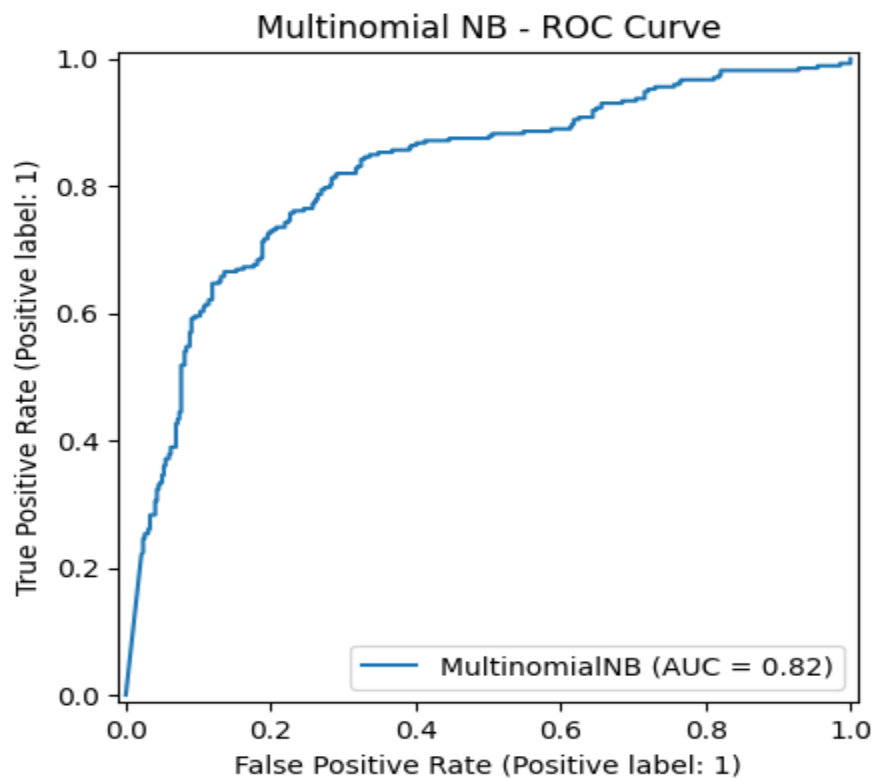
**Figure 10:** Confusion matrix for Ball Tree



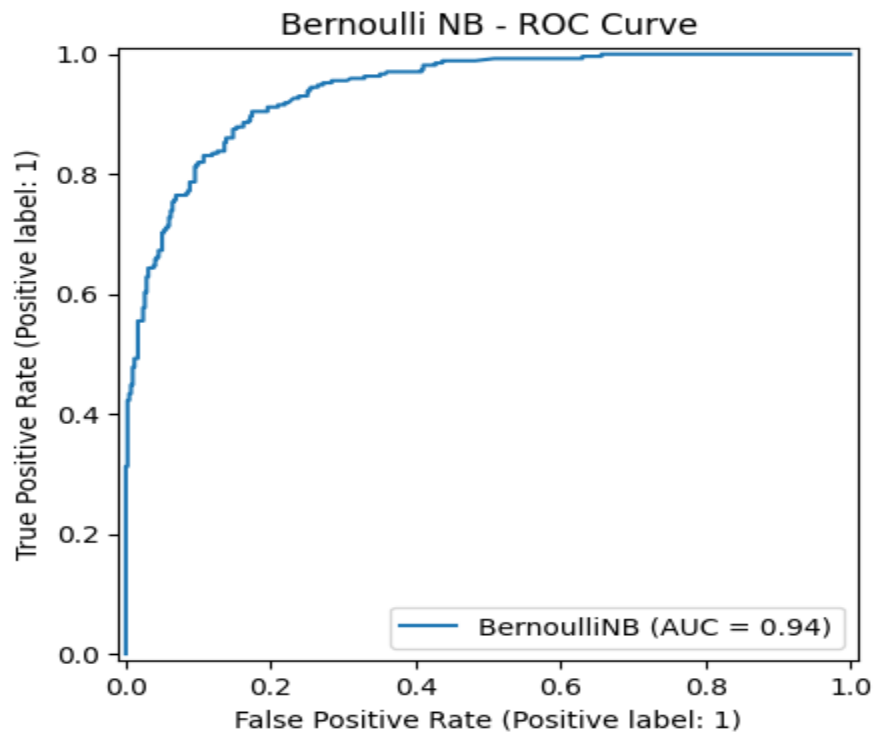
**Figure 11:** ROC curve for Gaussian NB



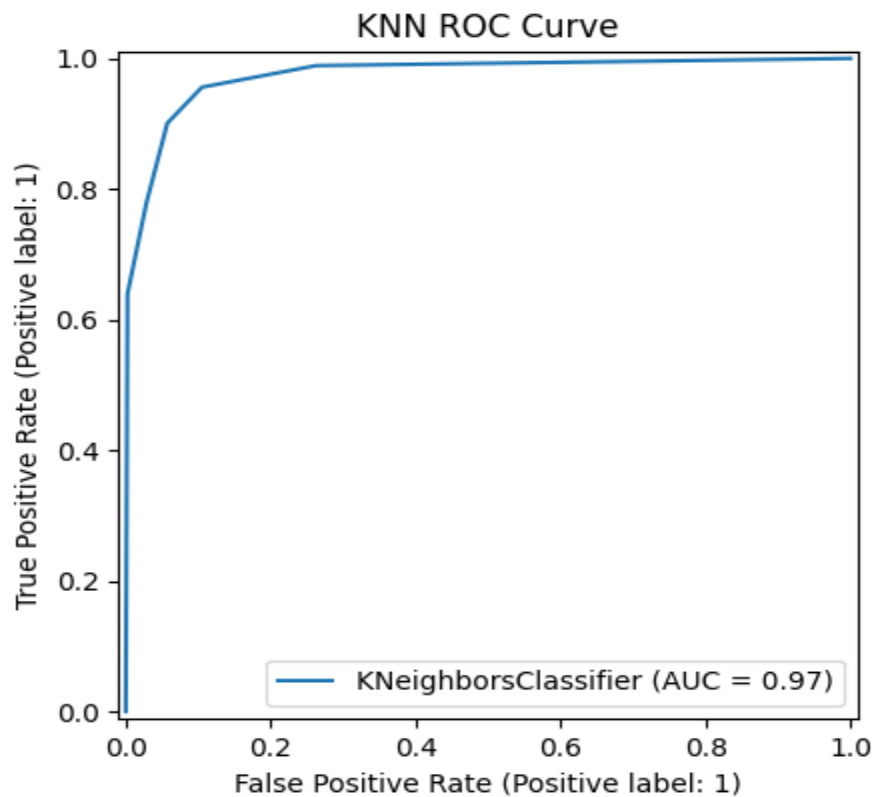
**Figure 12:** ROC curve for Multinomial NB



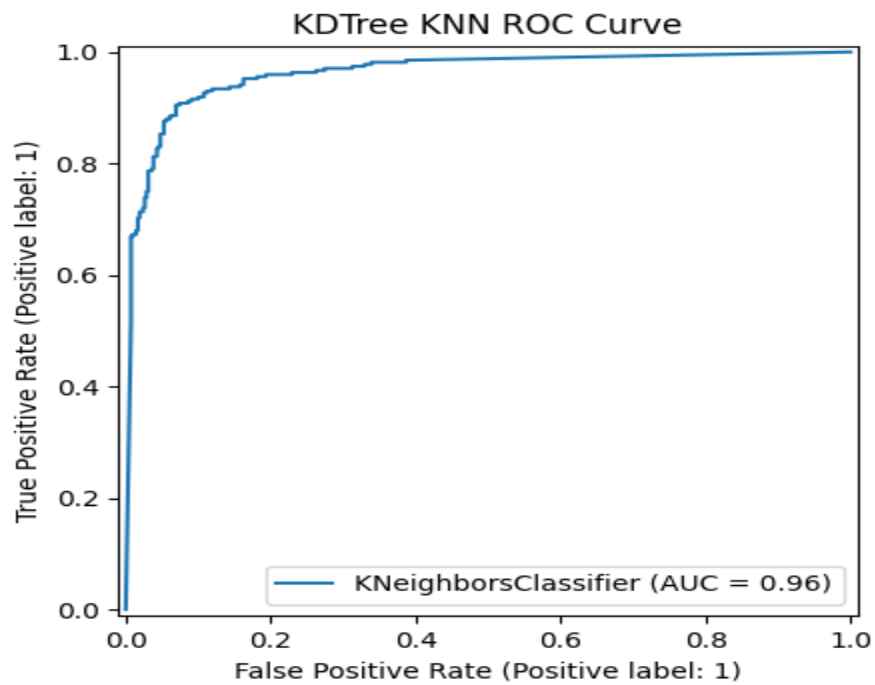
**Figure 13:** ROC curve for Bernoulli NB



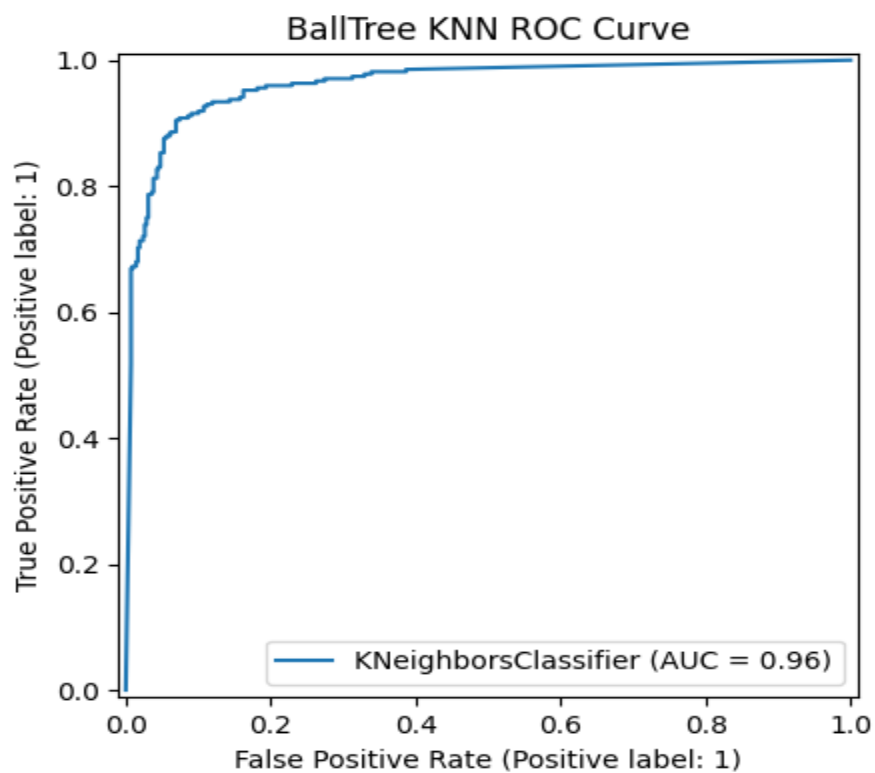
**Figure 14:** ROC curve for KNN



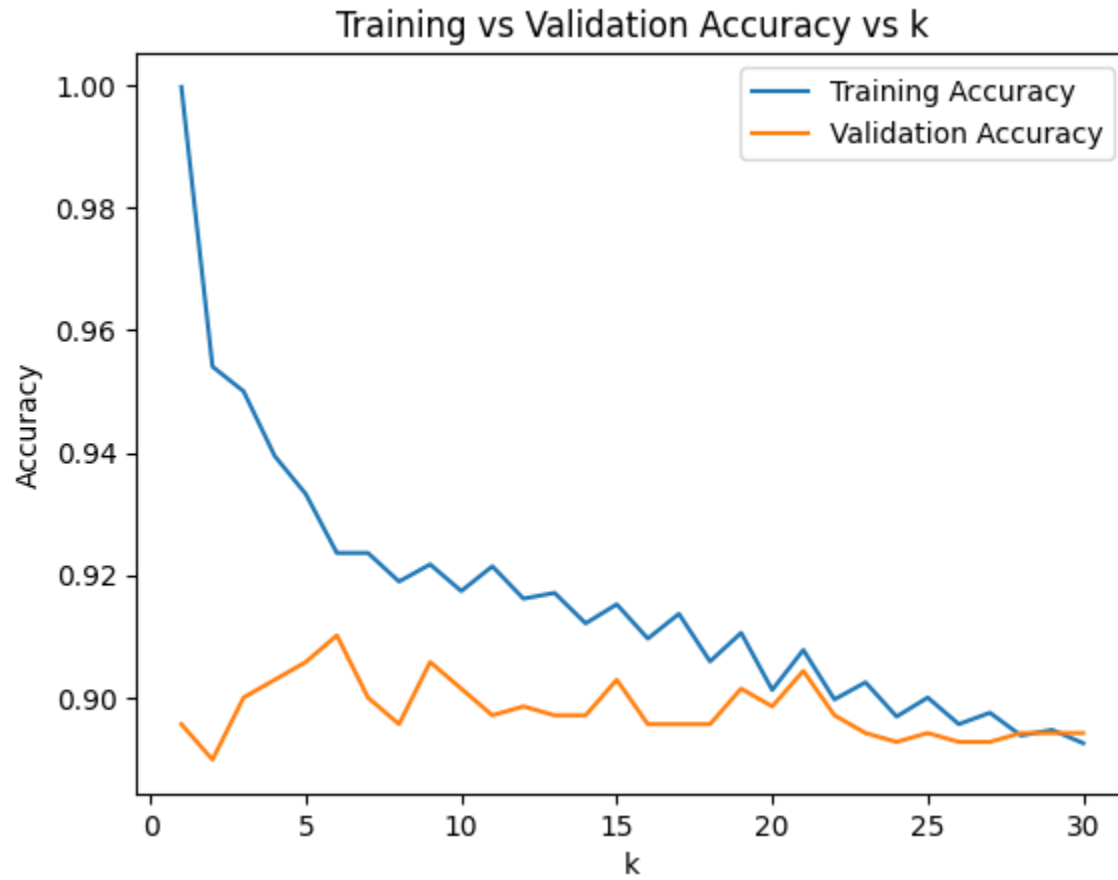
**Figure 15:** ROC curve for KD Tree



**Figure 16:** ROC curve for Ball Tree



**Figure 17:** Training vs Validation Accuracy vs k Plot for KNN



## 6. Performance Tables:

**Table 1:** Naïve Bayes Performance Metrics

| Metric            | Gaussian NB | Multinomial NB | Bernoulli NB |
|-------------------|-------------|----------------|--------------|
| Accuracy          | 0.8219      | 0.7641         | 0.8610       |
| Precision         | 0.7018      | 0.7104         | 0.8492       |
| Recall            | 0.9522      | 0.6764         | 0.7867       |
| F1 Score          | 0.8081      | 0.6930         | 0.8167       |
| Specificity       | 0.7374      | 0.8210         | 0.9093       |
| Training Time (s) | 0.0058      | 0.0041         | 0.0065       |

**Table 2: KNN Hyperparameter Tuning**

| Search Method     | Best k | Best CV Accuracy | Best Parameters                          |
|-------------------|--------|------------------|------------------------------------------|
| Grid Search       | 9      | 0.9183           | Weights: distance,<br>Algorithm: kd_tree |
| Randomized Search | 12     | 0.9161           | Weights: distance,<br>Algorithm: kd_tree |

**Table 3: KNN Performance using KDTree**

| Metric              | Value  |
|---------------------|--------|
| Optimal k           | 9      |
| Accuracy            | 0.9175 |
| Precision           | 0.9026 |
| Recall              | 0.8860 |
| F1 Score            | 0.8942 |
| Training Time (s)   | 0.0160 |
| Prediction Time (s) | 0.1036 |

**Table 4:** KNN Performance using BallTree

| Metric              | Value  |
|---------------------|--------|
| Optimal k           | 9      |
| Accuracy            | 0.9175 |
| Precision           | 0.9026 |
| Recall              | 0.8860 |
| F1 Score            | 0.8942 |
| Training Time (s)   | 0.0111 |
| Prediction Time (s) | 0.0972 |

**Table 5:** Comparison of Neighbor Search Algorithms

| Criterion           | KDTree       | BallTree      |
|---------------------|--------------|---------------|
| Accuracy            | 0.9175       | 0.9175        |
| Training Time (s)   | 0.0160       | 0.0111        |
| Prediction Time (s) | 0.1036       | 0.0972        |
| Memory Usage        | Low / Medium | Medium / High |

## 7. Overfitting and Underfitting Analysis:

### Difference between training and validation accuracy

When training accuracy is much higher than validation accuracy, the model is overfitting. When both accuracies are low, the model is underfitting. When both are high and close, the model generalizes well.

### **Effect of small and large values of k**

Small values of k cause overfitting due to high variance, while large values of k lead to underfitting because of increased bias. An intermediate value of k provides the best balance.

### **Role of hyperparameter tuning in generalization**

Hyperparameter tuning using cross-validation helps select optimal values of k and improves the model's ability to generalize to unseen data.

## **8. Bias–Variance Analysis:**

### **Bias behavior of Naïve Bayes**

Naïve Bayes has high bias because of its strong independence assumption but low variance, making it stable across datasets.

### **Variance behavior of KNN**

KNN shows high variance for small k and higher bias for large k.

### **Effect of tuning on bias–variance trade-off**

Tuning k balances bias and variance, resulting in improved generalization performance.

## **9. Observations and Conclusion:**

### **Observations**

- Naïve Bayes models trained very quickly and provided good baseline performance.
- KNN required more computational time, especially during prediction, due to distance calculations.
- Hyperparameter tuning significantly improved KNN accuracy and stability.
- KDTree showed faster prediction for lower-dimensional data, while BallTree handled complex distributions more efficiently.
- Training and validation accuracy plots helped identify optimal k values and avoid overfitting.

### **Conclusion**

This experiment demonstrated that both Naïve Bayes and KNN are effective for binary classification tasks. Naïve Bayes offered computational efficiency and stable performance, while tuned KNN models achieved higher predictive accuracy at the cost of increased computation. The final model selection was based on a balance between accuracy, generalization ability, and computational efficiency, making the optimized KNN classifier the preferred choice for this dataset.